

Foundations of Operations Research - Notes -
v0.6.0

260236

September 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

 [GitHub repository](#)

These notes are an unofficial resource and shouldn't replace the course material or any other book on foundations of operations research. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1	Introduction	5
1.1	What is Operations Research?	5
1.2	Decision-Making Problems	9
1.3	The Scheme of an Operations Research Study	15
	Index	22

Algorithms

1 Introduction

1.1 What is Operations Research?

Let's start with a definition of Operations Research (OR).

Definition 1: Operations Research (OR)

Operations Research (OR) is a branch of mathematics that uses **mathematical models** and **quantitative methods** to analyze complex decision-making problems and find optimal or near-optimal solutions.

So **Operations Research helps us make better decisions.** Suppose we have a real-world problem such as:

- Which machine should execute each job?
- How should a company allocate limited resources?
- What is the shortest route from A to B?
- How many workers should be scheduled?
- Where should a facility be built?

In all these cases, there are many possible decisions. Some are infeasible, some are feasible but bad, and some are better than others. OR tries to make this decision process **systematic and quantitative**.

In general, the idea is simple:

1. Real-world problem.
2. Formulate a mathematical representation.
3. Analysis.
4. Decision-making to find the best decision.

Example 1: Manufacturing Decision Problem

Imagine a company deciding how much of three products to manufacture. The real-world description may contain things like: limited workforce, limited machine hours, profit per product, production times.

OR translates those facts into mathematics. We might introduce variables such as:

x_1 = number of units of product 1,

x_2 = number of units of product 2,

And then construct an objective such as:

$$\max 5x_1 + 8x_2$$

Subject to resource constraints.

From the above example, we can see that OR is not simply “*doing calculations*”. A large part of OR is deciding **how to represent the decision problem mathematically in the first place**. To study decision problems, we have three major quantitative approaches:

- **Optimization** (*what should we do?*): we have decision variables, constraints, and an objective function, and we want the best feasible solution:

$$\min \text{ or } \max f(x) \quad \text{s.t. } x \in X$$

For example, minimize transportation cost while satisfying demand.

- **Game Theory** (*what should we do when others are also making strategic decisions?*): we have **multiple decision-makers** whose choices interact. The outcome for one player depends on what the others do. For example, two competing companies choosing prices.
- **Simulation** (*what will probably happen if we do this?*): we build a model of how a system behaves and then experiment with it. For example, simulate customers arriving at a call center to estimate waiting times under different staffing levels.

In these notes, we will focus on **optimization**.

🔗 Optimal vs Near-Optimal solutions

In the previous definition, we mentioned that OR finds **optimal or near-optimal solutions**. So the natural question is: “*why not always optimal?*”. If we have an optimization problem:

$$\min_{x \in X} f(x)$$

An **optimal solution** x^* satisfies:

$$f(x^*) \leq f(x) \quad \forall x \in X$$

It is literally the best feasible solution. But some **real optimization problems are computationally extremely difficult**. If the set of alternatives is enormous, finding the true optimum may **take too much time**. Then we may instead accept a solution that is:

- Feasible;
- Computationally obtainable;
- Very close to the best known solution.

That is a **near-optimal solution**. Later in the notes this distinction will become much more important, especially when we discuss combinatorial and integer optimization.

OR as an interdisciplinary field

Operations Research sits at the intersection of several areas: Applied Mathematics, Computer Science, Economics, and Industrial Engineering. Each contributes something different:

- **Applied mathematics.** Mathematics gives us the language used to describe the problem. For example:

$$\min f(x) \quad \text{s.t. } g_i(x) \leq 0$$

It also gives us the theoretical tools to determine whether solutions exist, whether they are optimal, and what properties they have.

- **Computer science.** A mathematical model is not enough. Suppose a problem has 2^{100} possible solutions. Writing the problem mathematically is easy; checking all solutions is impossible in practice. We therefore need algorithms to find good solutions efficiently.

This is where computer science enters:

$$\text{Model} \longrightarrow \text{Algorithm} \longrightarrow \text{Computer implementation}$$

These notes will repeatedly follow exactly this pattern. For example, later we will see shortest-path algorithms, minimum spanning-tree algorithms, maximum-flow algorithms, Simplex, and Branch-and-Bound.

- **Economics.** Many decision problems involve quantities such as: profit, cost, scarce resources, prices, trade-offs. So economic reasoning often determines what the objective function should represent. For instance:

$$\max \text{ profit} \quad \text{or} \quad \min \text{ operating cost}$$

Later, LP duality will even give an explicit economic interpretation to certain mathematical quantities.

- **Industrial Engineering.** Industrial engineering provides many of the real problems that OR was designed to solve: scheduling, logistics, transportation, production planning, inventory, facility location, and supply chains.

So OR is not mathematics developed in isolation. It is **mathematics designed around real decision problems**.

Key Takeaways 1

- **Operations Research (OR)** uses mathematical models and quantitative methods to support complex decision-making and find **optimal or near-optimal solutions**.
- Its overall goal is to **help make better decisions** by translating a real-world problem into a mathematical model that can be analyzed and solved.

- The main quantitative approaches introduced in OR are:
 - **Optimization**: find the best feasible decision;
 - **Game Theory**: study interacting decisions of multiple decision-makers;
 - **Simulation**: evaluate how a system behaves under alternative decisions.
- A **near-optimal solution** may be accepted when finding the true optimum is computationally too expensive.
- OR is interdisciplinary, combining **applied mathematics, computer science, economics, and industrial engineering**.

1.2 Decision-Making Problems

A **Decision-Making Problem** means choosing one feasible alternative among many, according to one or more criteria. It contains three main ingredients: alternatives, feasibility conditions, and criterion.

Suppose we have a set of possible decisions S . Not every decision may be allowed, so only some belong to the **feasible set** $X \subseteq S$. Then we judge feasible solutions according to some criterion (e.g., minimize cost, minimize time, maximize profit, etc.). So conceptually:

Choose $x \in X$ so that x is as good as possible

The **difficulty** is often that there are **too many alternatives**. That is why we need models and algorithms instead of trying possibilities one by one (brute force).

Assignment Problem

Suppose there are m **jobs**:

$$J_1, \dots, J_m$$

And m **machines**:

$$M_1, \dots, M_m$$

Every job can be executed by every machine. Let t_{ij} be the **execution time** of job J_i on machine M_j . For example:

	M_1	M_2	M_3
J_1	2	6	3
J_2	8	4	9
J_3	5	7	8

The decision is “*which job should be assigned to which machine?*”. But there are **constraints**:

- Every job must be assigned to **exactly one machine**;
- Every machine must be assigned to **exactly one job**.

And the **objective** is to **minimize the total execution time**.

Example 2: $m = 3$

For $m = 3$, one possible assignment is:

$$J_1 \rightarrow M_1, \quad J_2 \rightarrow M_2, \quad J_3 \rightarrow M_3$$

With total time:

$$t_{11} + t_{22} + t_{33} = 2 + 4 + 8 = 14$$

Another possible assignment is:

$$J_1 \rightarrow M_3, \quad J_2 \rightarrow M_2, \quad J_3 \rightarrow M_1$$

With total time:

$$t_{13} + t_{22} + t_{31} = 3 + 4 + 5 = 12$$

So the second one is better.

❓ Why is this an OR problem? Because the number of feasible assignments grows very quickly. With m jobs and m machines, there are $m!$ possible assignments exist. For example, with only $m = 10$ jobs and machines, there are already:

$$10! = 3'628'800 \text{ possible assignments}$$

So even a fairly small instance already has millions of possibilities. Therefore, a compactly described problem can have an enormous solution space. That theme will appear everywhere in OR.

⚠️ Network design problem

We represent the system with a graph

$$G = (N, E)$$

Where:

- N is the set of nodes;
- E is the set of possible links.

Each edge:

$$\{i, j\} \in E$$

Has a cost c_{ij} . The decision is: *which links should we build?* The requirements are:

- All nodes must ultimately be connected;
- Total cost should be as small as possible.

So we want to select a subset $E' \subseteq E$, such that the resulting network remains connected and:

$$\sum_{\{i,j\} \in E'} c_{ij}$$

Is minimized. This is the problem that will later lead to the Minimum Spanning Tree.

❓ But, why is it difficult? If there are $m = |E|$ possible edges, each edge can conceptually be: *selected* or *not selected*. Therefore there are up to 2^m possible subsets of edges. For $m = 50$:

$$2^{50} \approx 1.13 \times 10^{15}$$

Of course, many of these subsets are infeasible because they do not connect all nodes. But brute-force enumeration is still hopeless. So, again, we have a **huge search space**, and we **need algorithms to find good solutions**.

▲ Shortest path problem

We again use a graph, but the **decision is different**. Suppose the graph represents a road network. **Nodes** represent **locations**, and each **arc** has a **quantity** such as: distance, travel time, monetary cost. Given an origin s and destination t , the problem is to **find the minimum-cost path from s to t** .

For example, if we have two possible paths:

1. $s \rightarrow a \rightarrow t$
2. $s \rightarrow b \rightarrow c \rightarrow t$

We compute their costs by summing the arc costs. If:

$$c(s, a) + c(a, t) = 12$$

While:

$$c(s, b) + c(b, c) + c(c, t) = 8$$

The second path is preferable.

The important point is that the **criterion need not literally be distance**. A “*shortest*” path can mean: distance, travel time, or monetary cost. So mathematically, “*shortest*” really means “**minimum total arc weight**” according to whatever quantity the weights represent.

Already notice that all three problems fit the same OR pattern:

- **Assignment**
 - **Decision**: job-machine pairs
 - **Feasibility**: one job per machine and vice versa
 - **Objective**: minimize total execution time
- **Network design**
 - **Decision**: which edges to select
 - **Feasibility**: all nodes connected
 - **Objective**: minimize total edge cost
- **Shortest path**
 - **Decision**: which path to follow
 - **Feasibility**: valid path from source to destination
 - **Objective**: minimize path cost

That abstraction is extremely important. At first the applications look unrelated, but OR looks underneath the story and asks: “*what are the decisions? What makes them feasible? What do we optimize?*”.

 Example 3: Personnel scheduling

Suppose a hospital must prepare a weekly work schedule. The decision concerns things such as:

- Who works;
- On which day;
- During which shift.

The hospital has staffing requirements. For example, “*at least 5 nurses on Monday morning*”. It may also have constraints such as:

- Maximum weekly hours;
- Rest periods;
- Employee availability.

The objective might be to minimize the number of employees required, while still satisfying all staffing requirements.

Again:

- **Decision:** schedule of employees;
- **Constraints:** coverage and employment rules;
- **Objective:** use as few resources as possible.

This kind of problem becomes a classic scheduling/integer optimization problem.

 Example 4: Service management

Imagine a bank, airport, supermarket, or public office. At each moment, management must decide “*how many service counters should be open?*”.

Opening more counters improves service but costs more. Opening too few counters reduces staffing cost but increases waiting time. We might impose a service requirement such as:

$$\text{average waiting time} \leq 5 \text{ minutes}$$

And try to minimize the number of open desks. Usually, the trade-off is between lower operating cost and better service quality.

This example is slightly different from the earlier ones because quantities such as customer arrivals and waiting times may involve **uncertainty**. That will later be one reason why simulation can be useful.

🔗 Multicriteria decisions

So far, most examples had one clear objective: minimize cost, minimize time, or maximize profit. But many **real problems have several objectives**.

For instance, consider choosing a laptop. We might care about price, weight, performance. The difficulty is that these objectives can conflict.

- Laptop A may be cheaper, heavier and slower.
- Laptop B may be more expensive, lighter and faster.

Then there may be no universally “*best*” laptop. The decision depends on how much we value each criterion. This is called a **multicriteria** or **multi-objective problem**.

Conceptually, instead of one **objective function** $f(x)$, we may have **several**:

$$f_1(x), f_2(x), \dots, f_k(x)$$

For example:

$$\min f_1(x) = \text{price}, \quad \min f_2(x) = \text{weight}, \quad \max f_3(x) = \text{performance}$$

Later we will see ways of reducing this to a single optimization model.

🔗 Example 5: Maximum clique as a decision problem

This is another graph problem, but unlike shortest path or spanning tree, it is **computationally much harder**.

Suppose we have a social network represented by

$$G = (N, E)$$

Each node is a person. An edge means two people are connected. A **Clique** is a **subset of nodes in which every pair is connected**. For example, if:

$$\{A, B, C, D\}$$

Is a clique, then every pair must have an edge:

$$\{A, B\}, \{A, C\}, \{A, D\}, \{B, C\}, \{B, D\}, \{C, D\}$$

So a clique represents a group where everyone is directly connected to everyone else. The **Maximum Clique Problem** is to **find a clique containing the largest possible number of nodes**. The objective is therefore:

$$\max |\text{clique}|$$

One application is identifying highly interconnected communities in social networks.

❓ **Why is this example especially interesting?** Because the problem looks simple to state, but it is algorithmically difficult. And this is

an important OR theme: **some problems are easy to describe but hard to solve**. Shortest path and maximum clique are both graph problems, but their computational difficulty is radically different. Later, when complexity enters the course, maximum clique becomes a classic example of a hard combinatorial optimization problem.

✓ Key Takeaways 2

- A **decision-making problem** consists in choosing a feasible solution among many alternatives according to one or more criteria. Its fundamental ingredients are:

Decisions + Feasibility conditions + Objective/criteria

- Different real-world problems share the same OR structure:
 - **Assignment**: assign jobs to machines minimizing total execution time;
 - **Network design**: select links that connect all nodes at minimum total cost;
 - **Shortest path**: find an s - t path of minimum total arc weight.
- The number of alternatives can grow extremely quickly: an assignment problem with m jobs has $m!$ feasible assignments, while a network with m possible edges has up to 2^m subsets of edges. Hence, **enumerating all alternatives is generally not a viable solution method**.
- A criterion does not have to represent only monetary cost: it may represent **time, distance, profit, service quality**, or several conflicting quantities in a **multicriteria problem**.
- A **problem** can be **easy to state but difficult to solve**. The shortest-path and maximum-clique problems are both graph problems, yet their computational difficulty is very different.

1.3 The Scheme of an Operations Research Study

A standard workflow of an Operations Research project is as follows:

Problem $\rightarrow$ Model $\rightarrow$ Algorithm $\rightarrow$ Implementation $\rightarrow$ Analysis of results

The important point is that OR is not just “*solve a formula*”. Most of the intellectual work often happens **before** the algorithm is run. Let us go through the steps one by one.

1. **Step 1: Define the real problem.** We start from a real decision situation. For example: “*a company must decide how many units of each product to manufacture*”. At this stage, the problem is usually described in natural language. We need to identify things such as:

- What decisions must be made;
- What the objective is;
- What resources are limited;
- What requirements must be respected;
- What data are available.

This step is crucial because if we misunderstand the real problem, then even a perfectly solved mathematical model may be useless. So the first question is: “*What exactly are we trying to decide?*”.

2. **Step 2: Build the mathematical model.** Now we translate the real problem into mathematics. Typically we identify: **decision variables**, an **objective function**, and **constraints**.

For example, suppose:

x_1 = number of units of product 1, x_2 = number of units of product 2

If profits are 5 and 8, we may write:

$$\max 5x_1 + 8x_2$$

If total available production time is 100, we might have:

$$2x_1 + 4x_2 \leq 100$$

The real problem has now become an **optimization model**. So:

natural language $\rightarrow$ mathematical language

This modeling step is one of the main skills the FOR course aims to teach.

3. **Step 3: Select or develop an algorithm.** Once the mathematical model is known, we ask: “*What kind of optimization problem is this?*” The answer determines which algorithm is appropriate. For example:

Shortest path $\rightarrow$ shortest-path algorithm

Minimum spanning tree $\rightarrow$ Prim/Kruskal-type algorithm

Linear Programming $\rightarrow$ Simplex

Integer Linear Programming $\rightarrow$ Branch-and-Bound

So the model acts as a bridge between the application and the algorithm. The same algorithm can solve many completely different real-world problems if they have the same mathematical structure. That is one of the major strengths of OR. For example:

road routing and packet routing in a network

May both reduce to a shortest-path problem. The physical interpretation is different, but mathematically the structure is the same.

4. **Step 4: Implement or use software.** For realistic instances, solving the model manually is generally impossible. Therefore we implement the algorithm or use an existing solver. In this course, this will often mean something like:

Mathematical model $\rightarrow$ Python representation $\rightarrow$ CPLEX

The solver computes a numerical solution. But this does **not** mean the human work is finished. A solver only solves the model that we give it. If the model is wrong, then correct optimization of the wrong model is still wrong. This is an important practical principle.

5. **Step 5: Analyze the results.** The solver returns a solution to the mathematical model. But we must still interpret the solution in terms of the real problem. For example, if the solver returns:

$$x_1 = 40, \quad x_2 = 10$$

We must ask:

- *What does this mean in the real problem?*
- *Is the solution realistic?*
- *Are the assumptions reasonable?*
- *Is it implementable?*
- *Does it behave sensibly if parameters change?*

So the output must be translated back:

mathematical solution $\rightarrow$ real decision

Mathematical models as simplified representations

A mathematical model is **not the real system itself**. It is a simplified representation of the aspects of reality that matter for the decision. This distinction is fundamental.

Suppose we want to optimize delivery routes. The real system contains enormous detail: weather, traffic, driver behavior, vehicle reliability, road closures,

customer preferences, fuel prices, parking availability, etc. A useful model **cannot necessarily include everything**. Instead, we might retain only locations, distances, travel times, vehicle capacities, and delivery demands. So we perform an abstraction:

$$\text{Complex reality} \longrightarrow \text{Relevant mathematical structure}$$

The goal is therefore **not** to construct the most detailed model possible. The **goal** is to construct a **model** that is **sufficiently realistic to capture the essential features of the real problem, but sufficiently simple to be solved**. There is **always a trade-off**.

❓ What must a model capture? To create a useful model, we identify two things:

- **Fundamental elements.** These are the **objects involved in the decision**. For a production problem, they might be products, resources, production quantities, and profits. For a routing problem, they might be nodes, roads, and distances. For scheduling, they might be employees, shifts, and demand requirements.
- **Relationship between the elements.** The model must also represent **how these quantities interact**. For example, production consumes resources, and assigning a worker to a shift contributes to demand coverage.

Mathematically, these relationships typically become **constraints**. For example:

$$2x_1 + 3x_2 \leq 100$$

May represent the relation between production decisions and available machine time. So **constraints are not arbitrary inequalities**. They **encode real-world relationships**.

🔍 Modeling necessarily involves assumptions. Suppose we write:

$$\text{travel time on road } (i, j) = 10 \text{ minutes}$$

In reality, travel time varies. The **model may therefore implicitly assume** that travel times are known and fixed because that simplification allows us to solve the problem using deterministic optimization.

Similarly, the production example may assume that everything produced can be sold. This eliminates uncertain demand from the mod

Assumptions are therefore necessary. The important question is not “*does the model perfectly reproduce reality?*”, because **no model does**. The important question is: “*are the simplifications acceptable for the decision we want to make?*”.

Feedback and iterative refinement of the model

The OR process is not normally:

$$A \rightarrow B \rightarrow C \rightarrow D$$

With no return. Instead it is **iterative**. A better representation is:

$$\text{Problem} \rightleftharpoons \text{Model} \rightleftharpoons \text{Algorithm} \rightleftharpoons \text{Implementation} \rightleftharpoons \text{Results}$$

At any stage, we may discover something that forces us to reconsider a previous stage.

Example 6: Feedback

Imagine we construct a staff-scheduling model and obtain the mathematically optimal solution. The solver says:

Employee A works 16 consecutive hours.

Perhaps the original model forgot a labor-rule constraint. Then the optimization algorithm is not the problem. The **model is incomplete**. We go back and add, for instance:

$$\text{maximum consecutive working hours} \leq 8$$

Then we solve again. So:

results $\rightarrow$ discover missing constraint $\rightarrow$ modify model $\rightarrow$ solve again

Example 7: The model is computationally too hard

Suppose we model the real system extremely accurately. But then solving the **model requires days of computation**. We might decide to simplify it, thus:

Algorithm/implementation difficulty $\rightarrow$ model redesign

This illustrates an important **OR principle**:

model quality and computational tractability must be balanced

A perfect model that cannot be solved may be less useful than a slightly simplified model that can provide a good decision quickly.

 **Validation.** A central part of this feedback loop is **validation**. After obtaining a solution, we ask whether the model behaves consistently with reality.

For example:

- *Does the solution make operational sense?*
- *Does it violate an implicit requirement?*
- *Are the model parameters accurate?*
- *Do small changes in data create absurdly large changes in the solution?*

If problems appear, the model is refined. So an OR study often looks like:

1. Understand the problem
2. Model it
3. Solve it
4. Interpret and validate
5. Refine if necessary
6. Repeat

In summary, there are 3 different levels that we should not confuse:

Problem $\neq$ Model $\neq$ Algorithm

Where:

- The **problem** is the real decision situation.
- The **model** is its mathematical representation.
- The **algorithm** is the method used to solve that mathematical representation.

For example:

- “*Find the cheapest road network*” is the real problem.
- “*Minimum spanning tree problem on $G = (N, E)$* ” is the mathematical model.
- “*Kruskal’s algorithm*” is one possible solution method.

Key Takeaways 3

- A **mathematical model** is a **simplification of reality**, not a complete reproduction of the real system.
- A good model keeps the **fundamental elements** of the decision problem and the **relationships** among them.
- Real-world relationships are translated into mathematical structures, especially **constraints**.
- Every model relies on **assumptions**; the important question is whether those assumptions are reasonable for the decision being

made.

- There is always a trade-off between **realism** and **solvability**:

$$\text{model accuracy} \leftrightarrow \text{computational tractability}$$

- An OR study is **iterative**, not a one-way pipeline. Results may reveal missing constraints, wrong assumptions, inaccurate parameters, or excessive computational complexity.
- **Validation** checks whether the obtained solution is meaningful and consistent with the real system.
- The model should be **refined and solved again** whenever validation reveals problems.
- Always distinguish:

$$\text{Problem} \neq \text{Model} \neq \text{Algorithm}$$

Where the **problem** is the real decision situation, the **model** is its mathematical representation, and the **algorithm** is the method used to solve the model.

Thus, OR transforms a real decision problem into a solvable mathematical model, validates the result, and iteratively refines the model when necessary.

References

- [1] Braz Pascoal Marta Margarida. Foundations of Operations Research. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

C

Clique 13

D

Decision-Making Problem 9

M

Maximum Clique Problem 13

Multi-Objective Problem 13

Multicriteria Problem 13

O

Operations Research (OR) 5